Agibot D1 Ultra\Edu SDK Development Guide

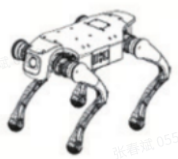
Chapter I: Product Introduction

1.Product Overview

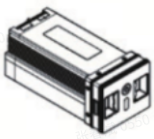
AgiBot quadruped robot D1 Ultra\Edu features 12 degrees of freedom and multiple motion gaits and actions. Additionally, this product offers an open-source SDK for motion control algorithm and communication protocols, enabling users to perform secondary development according to their specific requirements.

> 🐵 Note：The secondary development documentation and sdk of D1 Ultra and D1 Edu are completely identical. Please refer to this document as the authoritative source.

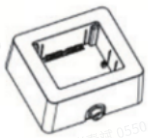
2.Product List



D1 x1(Battery not included)



battery x1



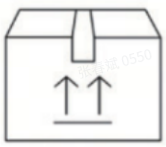
charging dock x1

adapter x1

Adapter power cord x1

remote control x1

Instruction Manual x1



shipping box x1

Anti pulley x4

toolkit x1

3. Product Parameter

Classification	Specification	Instruction
Basic Information	Material Information	Aluminum+High-Strength Engineering Plastics

	Standing Dimensions (length * width * height)	630mm*360mm*415mm
	Stowed Dimensions (length * width * height)	670mm*425mm*145mm
	Total Weight (including battery)	15kg
	Operating Temperature	0~40°C
	Protection Rating	IP54
	Charging Time	1h
	Battery Life	1~2h
	Maximum Range	6km
Performance Parameter	Basic Computing Power	8-core high-performance CPU
	Maximum Operating Power	3500W
	Maximum Speed	3.7m/s
	Maximum load	8kg(Maximum10kg)
	Continuous Stair Climbing Height	16cm
	Maximum Gradeability	Standard environment: 30 degrees; Extreme environment: 40 degrees
	Maximum Jump Height	35cm
Robot Joint parameters	Actuator Type Joint Range of Motion (Shank)	High Power Density Actuator
	Maximum Joint Torque	48N.M
	Joint Range of Motion (Torso)	-28°~28°
	Joint Range of Motion (Thigh)	-170°~66°
	Joint Range of Motion (Shank)	-35°~156°
Sensors	Camera FOV	DFO122°，HFO122°，VFO70°
	IMU	Stardard
Function List	Standard Motions	

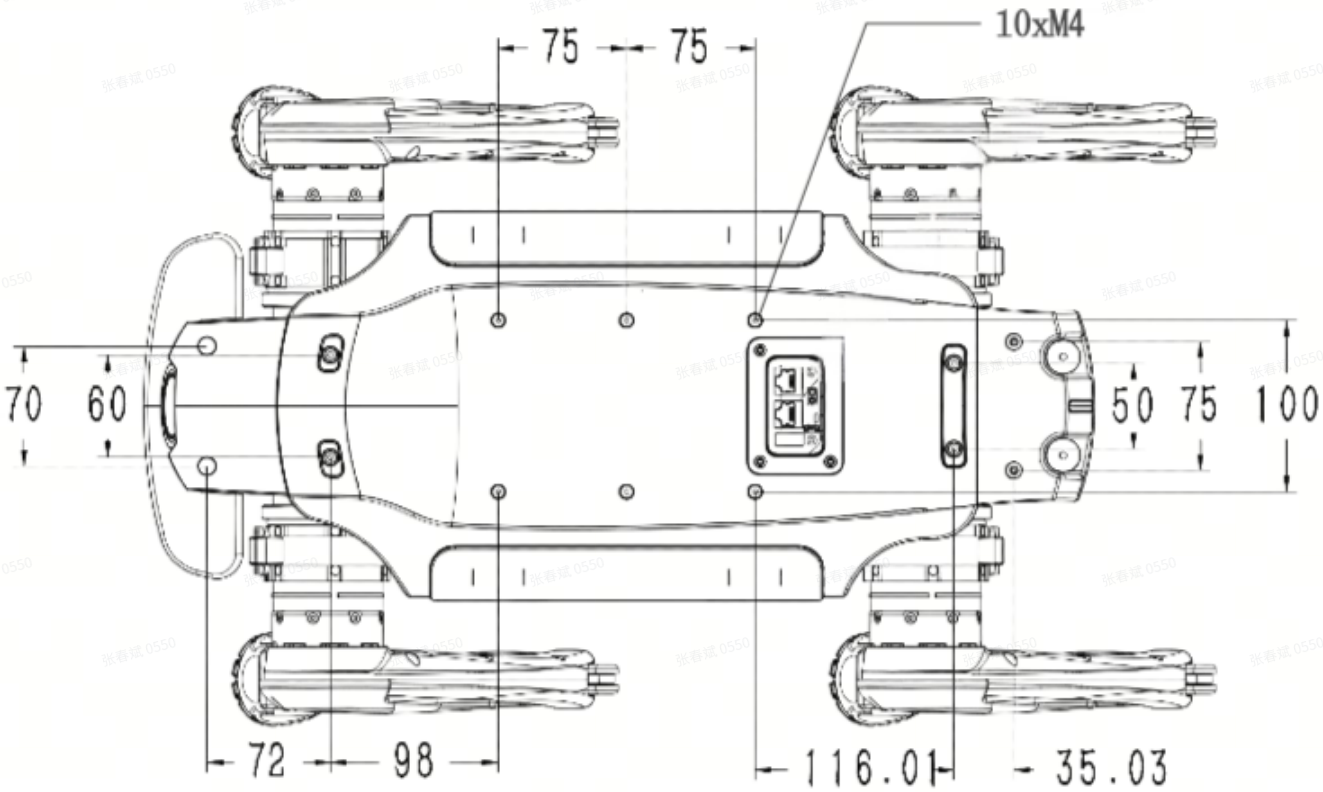
		Stand, Lie Down, Passive, Pitch/Posture Adjustment, Height Adjustment, Horizontal Turn
	Specialized Motions	Forward Jump, Backward Jump, two leg Stand, backflips, shaking hand, etc.
	Real-time Video Transmission	Supported
	OTA Updates	Supported
	Secondary Development	Supported
	Expansion Interfaces	Ethernet Port, USB Port, Power Port, SBUS Interface, UART Interface
Accessories	Adapter	Standard
	Charging Dock	Standard
	Handheld Remote Controller	Optional
	Remote Controller with Display	Optional
	EPP Trolley Case	Optional
Else	Warranty Period	1 year

👍 The charging time was measured under a standard ambient temperature of 25°C.

The maximum range were measured at a speed of 1 m/s under unloaded conditions.

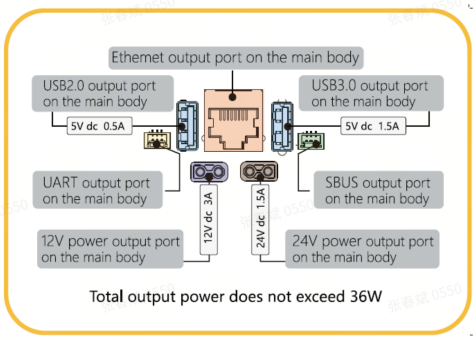
The above parameters are derived from laboratory test data. Actual performance may vary based on specific usage conditions, operational practices, and other factors. Please refer to real-world conditions for application.

4. Rear Load Expansion Mounting Hole Layout

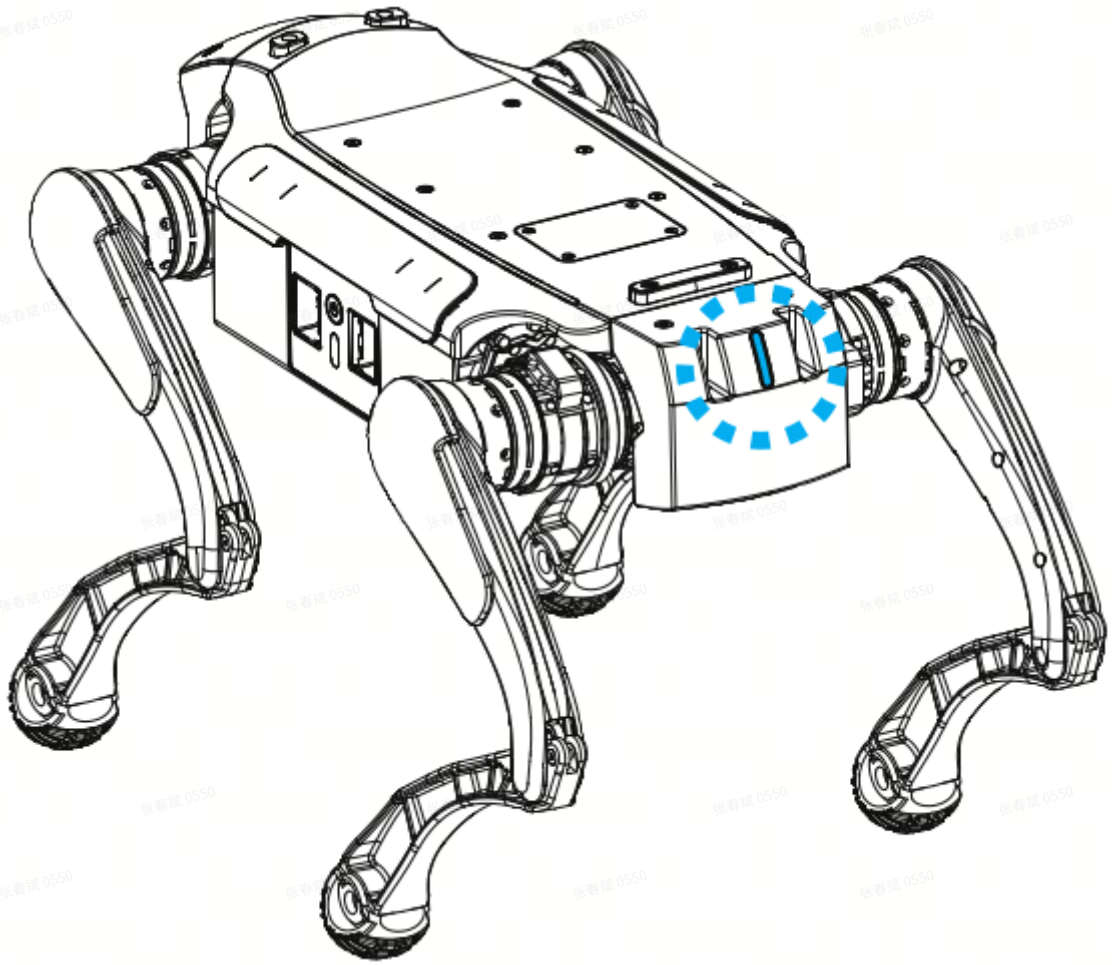


Note：The back load mounting hole position is M4

5. Interface

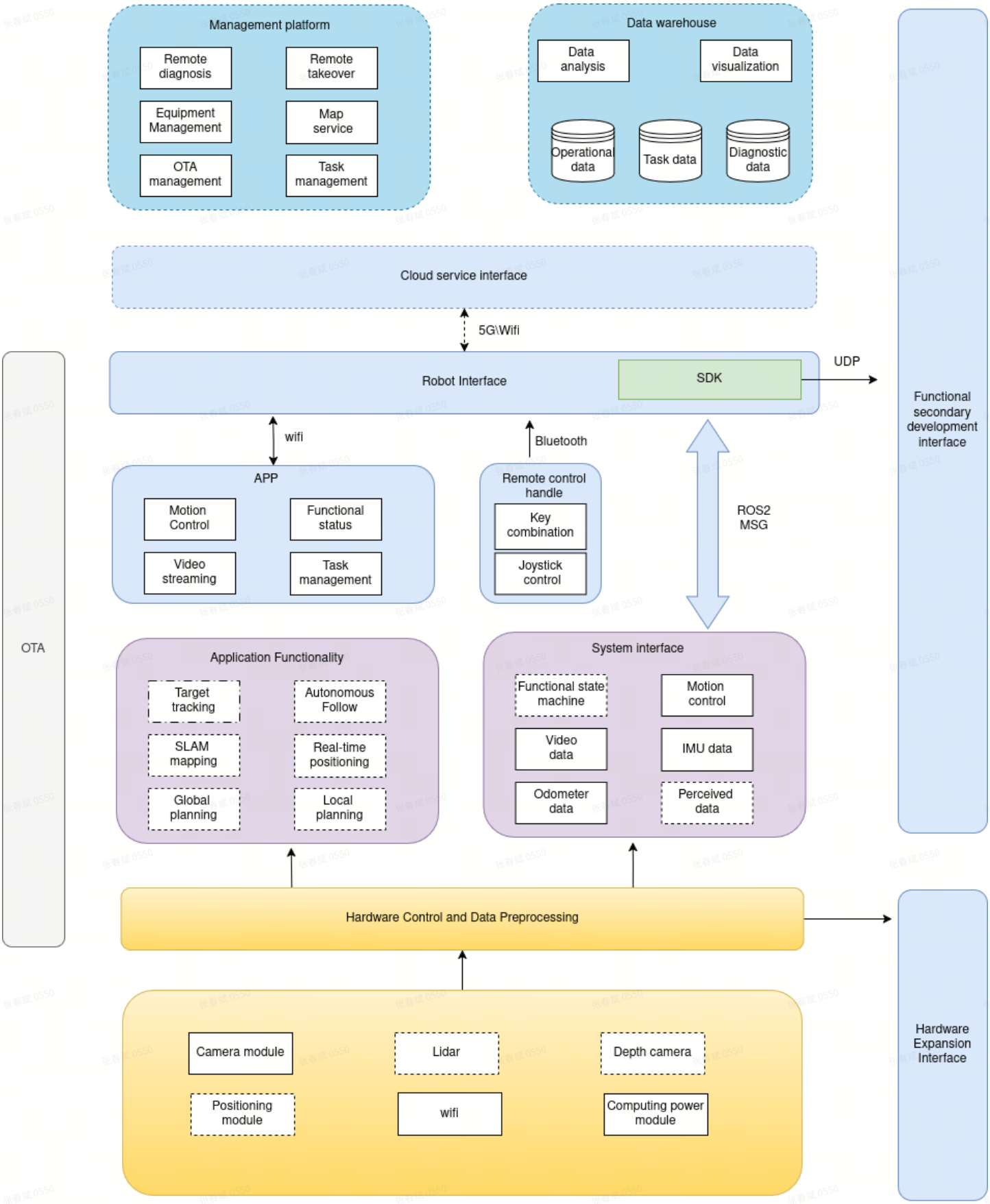
Serial Number	Interface name	Number	Instruction	Note
1	Ethernet Port	1	RJ45 port, supporting 1000 Mbps transmission rate, designed for high-speed data transfer. Enables network connectivity, remote monitoring, data transmission and other application.	
2	USB Port	2	Type_A port, used for connecting external storage devices, cameras, sensors, and other USB peripherals. Provides high-speed data transfer and power supply capabilities.	

3	Power Port	2	Standard DC power port, delivering stable power supply with 12V and 24V voltage outputs. Total output power does not exceed 36W (Note: Future versions will support up to 100W).
4	SBUS Port	1	Standard SBUS port, designed for connecting remote controller receivers or other communication devices supporting the SBUS protocol. Enables precise transmission of remote control commands.
5	UART Port	1	Standard UART port, used for connecting embedded systems, communication devices, or other peripherals supporting the UART protocol. Facilitates data exchange and control between devices via asynchronous serial communication.



Chapter II: System Architecture

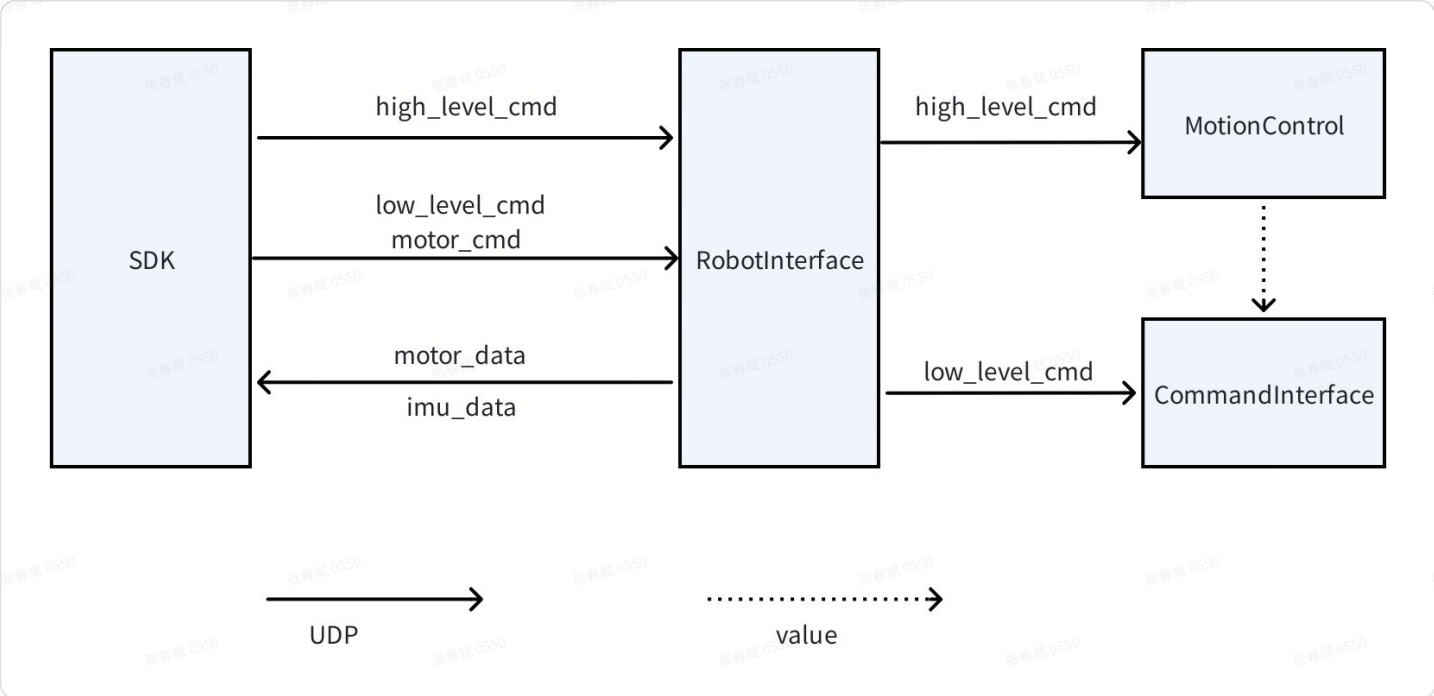
1. AGIBot System Architecture



Note: The solid line diagram represents the functionality that has been implemented
The dashed diagram represents functionality that can be achieved through secondary development

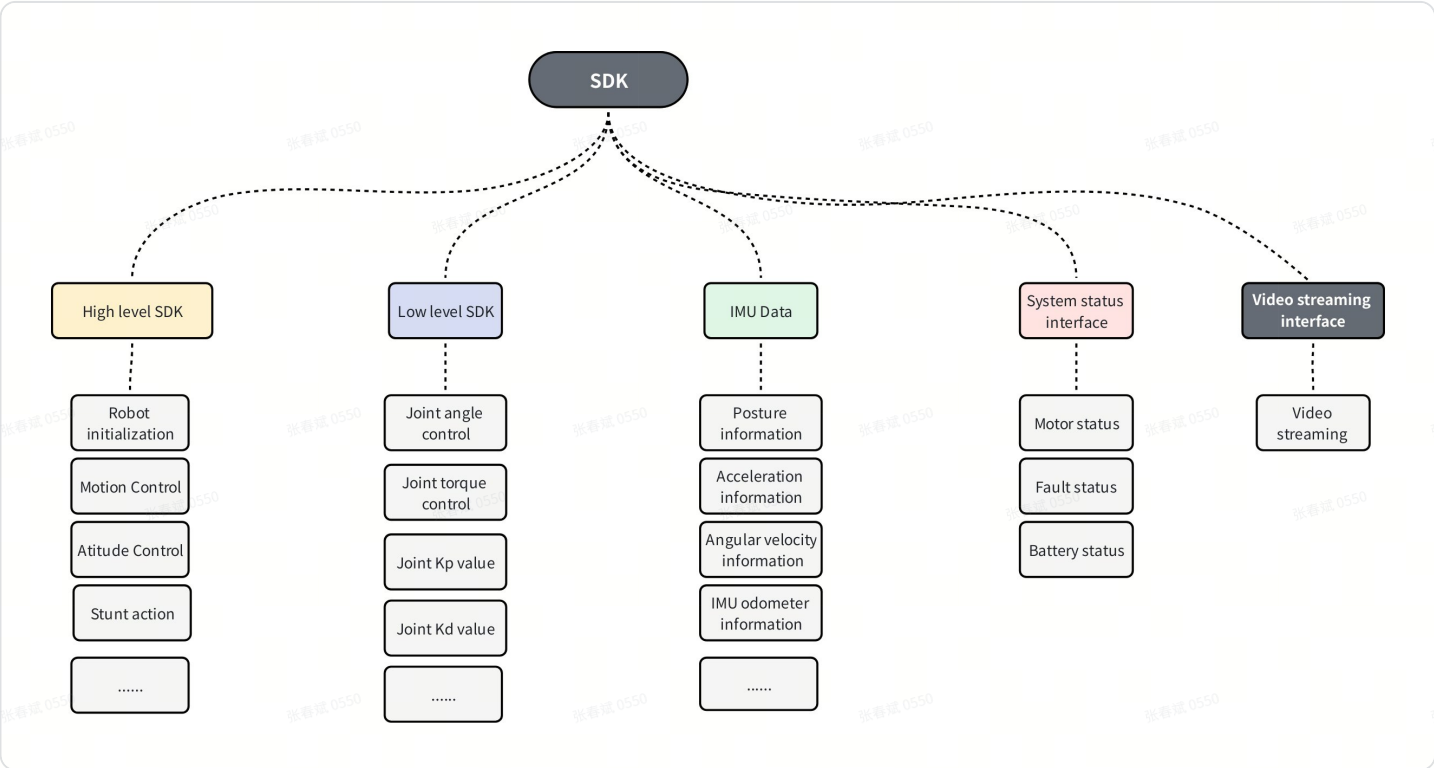
🌟 Note: Solid wireframes represent implemented functions, while dashed wireframes can be realized through secondary development.

2. SDK Control Diagram



3. SDK Interface Diagram

The SDK interfaces of motion control have been available, including App function service interface, motor control interface, IMU service interface, device state service interface and video stream data interface.



4. List of SDK Interfaces

Serial Number	Interface Name	Function	SDK Version
1	App Function Service Interface	Realize the motion control of the robot, including forward and backward movements, turning, lying down, rotation, two leg standing, etc.	V0.2.6
2	Motor Control Service Interface	Control the underlying interface of the motor, set the target motion state of the motor, including position, speed, torque, etc. By setting different parameters, users can control the motor to perform specific actions in different modes.	V0.2.6
3	Device State Service Interface	Query and obtain the current device status and system resource usage information	V0.2.6
4	IMU Service Interface	Obtain IMU original data information	V0.2.6
5	Video stream data interface	Send the video stream to the client.	V0.2.6

Chapter III: SDK Deployment Guide.

1.SDK Introduction

- This document introduces the Software Development Kit (SDK) for D1 Ultra\Edu, providing detailed information of service interfaces. These include the App function service interface, motor control interface, IMU service interface, device state service interface and video stream data interface.
- Each chapter separately introduces the invocation methods for different service interfaces, the standard format for data input/output, and some functional development examples.
- Please note that interface definitions will change along with the development status. Always refer to the latest version of this document.
- Communication between SDK and the onboard program follows a specific protocol and communication methods of different SDK versions may vary. Please select the SDK version that corresponds to your device version. If possible, it is recommended to update the onboard program to the latest version for better compatibility.
- All operational states are designated by the secondary developer when controlling the unit via the SDK interfaces. The onboard control system does not perform additional interventions or state transitions.
- Please ensure that the system has sufficient resources when running the SDK. Otherwise, the motion control module may fail.

2.Prerequisites

- ubuntu22.04
- Cmake 3.8+
- GCC 11+
- Eligen3
- Boost
- python3+

3. Robot Version Inquiry

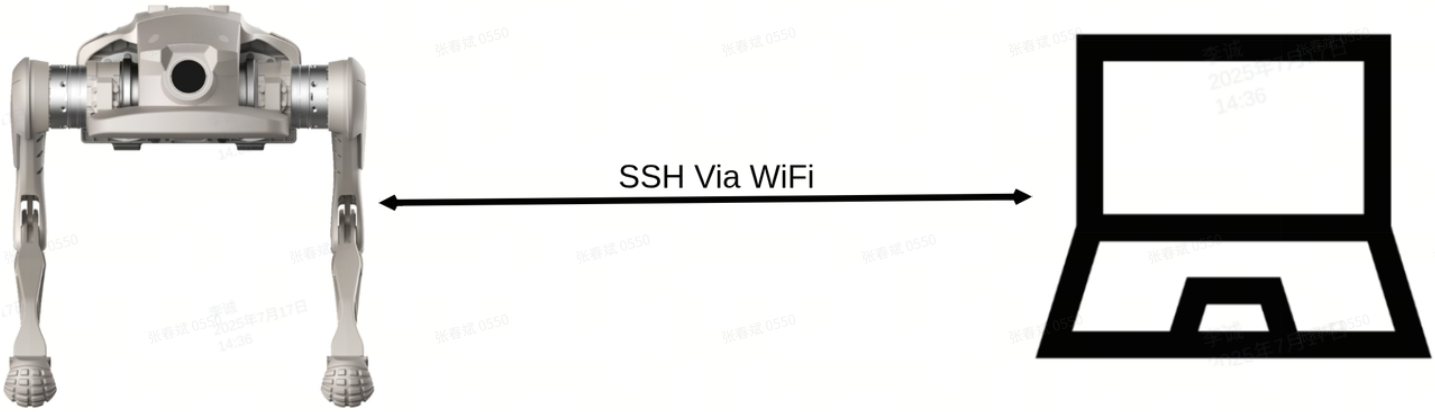
3.1 Device Login

The device is equipped with wireless and wired network interfaces. The wireless network information is located on the right side label of the device. The label contains the device SSID and password information. The default IP of the robot body is as follows:

Network Interface	IP	DHCP

Wireless Network	192.168.234.1	Yes
Wired Network	192.168.168.168	None

SSH login is provided. Users can log in via SSH after establishing a connection with the equipment via wireless or wired network.



- Wireless Connection

Code block

```
ssh firefly@192.168.234.1     #code:firefly
```

The wireless network has DHCP service. After connecting, please make sure that the device does not have a fixed IP configured. Then, communicate with the robot using 192.168.234.1.

- Wired Connection

Code block

```
ssh firefly@192.168.168.168    #code:firefly
```

The wired network does not have DHCP service. After connecting, users need to manually configure a static IP address. The IP address is within the 168 subnet.

3.2 Version Number Query of Remote Control

After logging into the device, execute the following command to view the current device version:

Code block

```
grep -oP 'motion-control_\K[^_]+' /etc/release/*[^rootfs]*.yaml
```

After executing the command, if the version number displayed is lower than 0.2.6, it indicates that the device firmware is outdated and requires an upgrade. Please contact after-sales support for assistance.

Based on the output code after command execution, refer to the table below to identify the corresponding version:

Remote Control Version Number	SDK version
0.1.7	0.1.5
0.1.8	0.2.0
0.2.6~latest	0.2.7

🎨 **This document is applicable to SDK version 0.2.6 and above, and motion control version 0.2.6 and above (robot version is 0.2.0 and above).**

4.Obtain SDK

Please contact the after-sales support staff or visit the following address:

```
git clone https://github.com/AgibotTech/agibot_D1_Edu-Ultra.git
```

5.Communication Configuration

"3.1 Device Login" has already been logged into the device via SSH. Based on the selected connection method, the following modifications will be made.

5.1.Modify the SDK communication configuration file

Through vim command, Modify `/opt/export/config/sdk_config.yaml` File:

```
target_ip: "<Local IP>"
target_port: 43988
```

- `target_ip`： The remote control address for the robot should be set to an IP on the same network segment as the development computer. Please ensure there is no IP conflict.(Can not be 192.168.234.1 or 192.168.168.168)
- `target_port`： 43988， Do not modify.

🌰 **After modifying the configuration parameters, the robot needs to be restarted.**

🏝️ **During remote control, the remote controller is not usable.**

🥖 **After the remote control is completed, please change the "target_ip" back to 127.0.0.1. Otherwise, the robot would not be remotely controlled.**

Plase refer to "5.2 Configure SDK_CLIENT_IP" when using network cable connection. For wireless connection, this step is not necessary.

5.2 Configurate SDK_CLIENT_IP

If you need to control devices using network cables or other IP addresses not in 192.168.234.X range, modify

`/opt/app_launch/start_motion_control.sh` file through the vim command：

```
#!/bin/bash

echo "start motion control"

SHM_FILE="/dev/shm/spline_shm"

while true; do
    if [ -e "$SHM_FILE" ]; then
        echo "The shared memory file $SHM_FILE is already exist。"
        break
    else
        echo "The shared memory file $SHM_FILE does not exist, retry after 1 second..."
        sleep 1
    fi
done

```

```
18  echo "The shared memory file is ready and can proceed with subsequent operations"
19
20  sudo ifconfig lo multicast
21  sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev lo
22
23  export LD_LIBRARY_PATH=/opt/export/mc/bin
24  export ROBOT_TYPE=P2
25  export SDK_CLIENT_IP="robot ip"
26
27  cd /opt/export/mc/bin && taskset -c 7 ./mc_ctrl r
```

On Line 27, add: `export SDK_CLIENT_IP="robot ip"`, and fill the robot's IP address with the corresponding parameter. Change the robot's IP to 192.168.168.168 when using wired networks. Just modify it to the corresponding IP if it is another IP.

🌰 **After modifying the configuration parameters, restart the robot.**

🏝️ **After configuring this parameter, there is a probability that the robot will be unable to use the remote control!**

After configuring `SDK_CLIENT_IP`, if the device's network fails to initialize at startup and the specified network interface remains without an assigned IP address, the motion control program will be unable to bind to the designated address. This binding failure will result in the robot being unresponsive to remote control commands.

🥖 **After the remote control is completed, please change the "target_ip" back to 127.0.0.1. Otherwise, the robot would not be remotely controlled.**

6.Modify SDK Demo configuration

6.1 Configurate python IP and port

`<working directory>/agibot_sdk/demo/zsl-1/python/examples/`, make modifications as needed.

Code block

```
app.initRobot("127.0.0.1", 43988, "127.0.0.1")    #local_ip, local_port, dog_ip
```

6.2 Configure C++ IP and port

`{working directory}/agibot_sdk/demo/zsl-1/python/examples/` make modifications as needed.

Code block

```
constexpr int CLIENT_PORT = 43988;   // local port
std::string CLIENT_IP = "127.0.0.1"; // local IP address
std::string DOG_IP = "127.0.0.1";    // dog ip
```

🎨 **The robot IP address and local port in the configuration parameters must match the target_ip and target_port parameters in sdk_config.**

Please ensure the robot's IP address and port number match; otherwise, communication will fail!

If the IP address in sdk_config.yaml is deleted or commented out, the machine's motion control service will not start automatically!

The configuration file will be overwritten and reset after device or program update. Relevant configurations must be reconfigured!

7.Run Demo Example

7.1 Compile Demo

In the SDK files on the local PC side

Code block

```
cd demo/zsl-1/cpp
mkdir build
cd build
cmake ..
make -j6
```

7.2 Run sdk demo

- Run Python demo

Code block

```
cd demo/zsl-1/python/examples
python highlevel_demo.py
or
python lowlevel_demo.py
```

- Run C++ demo

Code block

```
cd demo/zsl-1/cpp/build
./highlevel_demo
or
./lowlevel_demo
```

7.3 Call SDK

- Call SDK in Python

```
import os
import platform
import sys
arch = platform.machine().replace('amd64', 'x86_64').replace('arm64', 'aarch64')
lib_path = os.path.abspath(f'{os.path.dirname(__file__)}/../../../../lib/zsl-1/{arch}')
sys.path.insert(0, lib_path)

import mc_sdk_zsl_1_py
app  = mc_sdk_zsl_1_py.HighLevel()
app.initRobot("<local ip>",43988,"192.168.234.1")
app.standUp()
app.lieDown()
...
```

- Call SDK in Python Call SDK in cmake

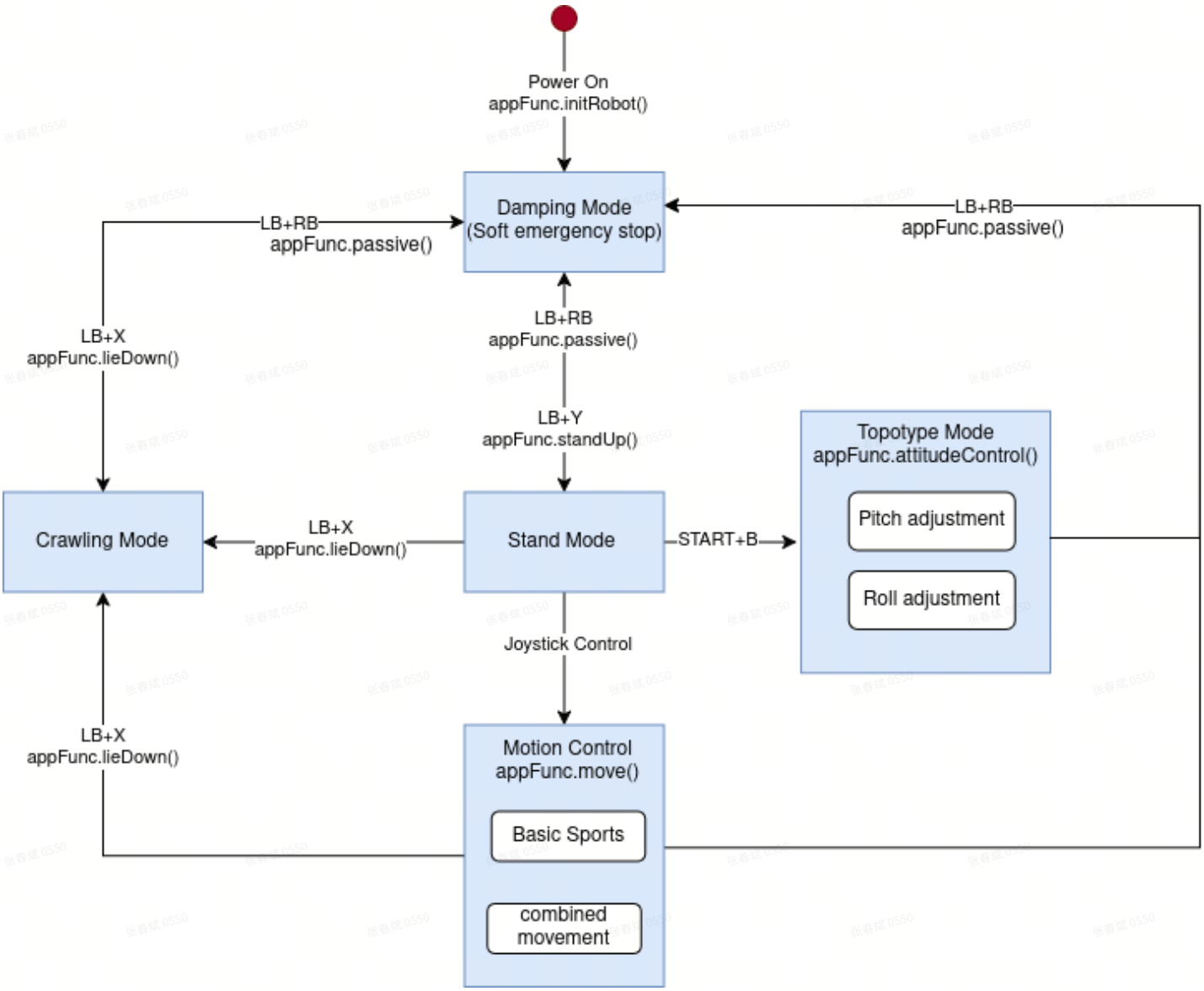
代码块

```
cmake_minimum_required(VERSION 3.16)

project(test)

set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_STANDARD 17)

set(ARCH "unknown")


if(CMAKE_HOST_SYSTEM_PROCESSOR MATCHES "x86_64|amd64")
    set(ARCH "x86_64")

elseif(CMAKE_HOST_SYSTEM_PROCESSOR MATCHES "aarch64|arm64")
    set(ARCH "aarch64")
else()
    message(FATAL_ERROR "Unsupported architecture: ${CMAKE_HOST_SYSTEM_PROCESSOR}, please check.")
endif()

message(STATUS "Detected system architecture: ${ARCH}")


set(LIB_PATH "${CMAKE_SOURCE_DIR}/../../../lib/zsl-1/${ARCH}")
message(STATUS "Link the library path: ${LIB_PATH}")

find_library(MC_SDK_LIB
    NAMES "mc_sdk_zsl_1_${ARCH}"
    PATHS ${LIB_PATH}
    NO_DEFAULT_PATH
    REQUIRED
)

if(NOT MC_SDK_LIB)
    message(FATAL_ERROR "In ${LIB_PATH} not find the library file mc_sdk_zsl_1_${ARCH}.so, please recheck.")
else()
    message(STATUS "Find the library file: ${MC_SDK_LIB}")
endif()

# highlevel
add_executable(highlevel_demo highlevel_demo.cpp)
target_link_libraries(highlevel_demo ${MC_SDK_LIB})
target_include_directories(highlevel_demo PRIVATE
    ${CMAKE_SOURCE_DIR}/../../../include
    ${CMAKE_SOURCE_DIR}/../../../include/zsl-1
)
```

```
# lowlevel
add_executable(lowlevel_demo lowlevel_demo.cpp)
target_link_libraries(lowlevel_demo ${MC_SDK_LIB})
target_include_directories(lowlevel_demo PRIVATE
    ${CMAKE_SOURCE_DIR}/../../../include
    ${CMAKE_SOURCE_DIR}/../../../include/zsl-1
)
```

8.Secondary Development Instructions

Command issuance must follow the state transition logic below; otherwise, it may result in abnormal situations such as the robot falling over, malfunctioning, or becoming unresponsive.



Joint Control Command

📌 Command Sequence

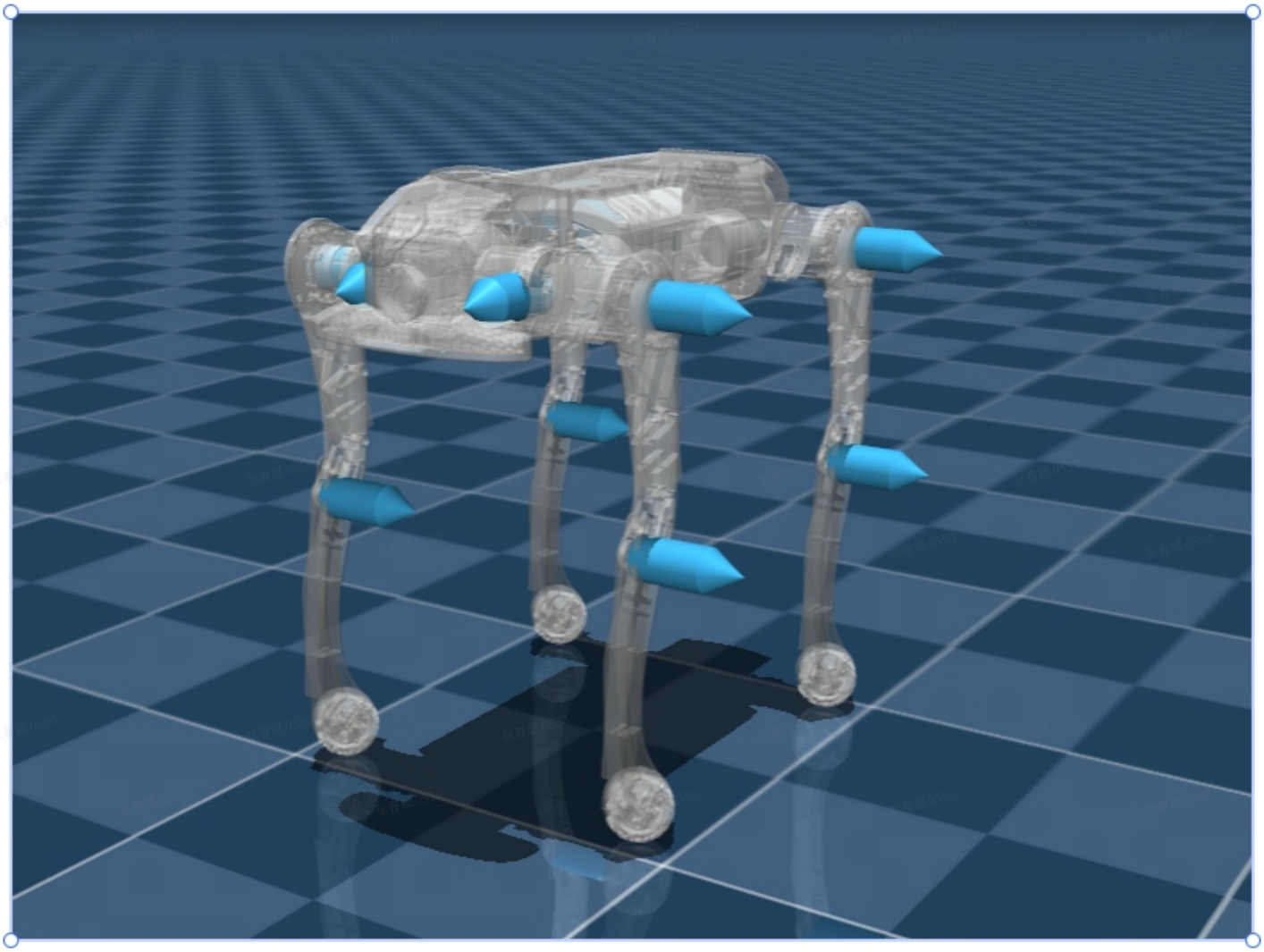
- FR（Front-Right）
- FL（Front-Left）
- RR（Rear-Right）
- RL（Rear-Left）

Joint Orientation Definition

A,H,KJoint Coordinate System：Forward(X), left(Y), Up(Z).

The coordinate directions follow the right-handed coordinate system, where forward is the positive X direction, left is the positive Y direction, and up is the positive Z direction.

Leg Order: Front-Right, Front-Left, Rear-Right, Rear-Left



Chapter IV: API Function Introduction

1.High-Level Function Introduction

This category includes methods for controlling the application functions of the robot. Users can use these methods for upper-level control and data acquisition.

> 🎨 Device status interface support frequency： HighLevel is 50hz
>
> The supported request frequency in motion control is 500 Hz. Request data must be sent at this rate to ensure stable device operation.

1.1. Initialization

Function Name	**initRobot(std::string & local_ip, int local_port, string& dog_ip = "192.168.234.1")**
Function Overview	To establish communication, the default IP is "192.168.234.1". If the robot's IP needs to be changed, the new IP must be passed through this interface.
parameter	local_ip:IP address of host, required. local_port:port of host，optional. dog_ip:IP address of robot, optional.
Return Value	None
Note	Failed to establish connection. Terminal output failure logs

1.2. Check the Connection Status

Function Name	**checkConnection()**
Function Overview	Check the connection status of robot
parameter	None
Return Value	Type：bool true:Connection successful false:Connection fail
Note	Can perform heart rate detection

1.3. Stand Up

Function Name	**standUP()**
Function Overview	Control the robot to stand up
parameter	None
Return Value	Type：uint32_t 0:normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	The robot cannot directly switch to this state while moving.

1.4. Lie Down

Function Name	lieDown()
Function Overview	Control the robot to lie down
parameter	None
Return Value	Type：uint32_t 0:Normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	The robot cannot directly switch to this state while moving.

1.5. Passive

Function Name	passive()
Function Overview	Control the robot to fall down
parameter	None
Return Value	Type：uint32_t 0:Normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	robot lie down.

1.6. Move

Function Name	move(float vx, float vy, float yaw_rate)
Function Overview	Control the robot to move
parameter	vx:Forward velocity (required). Pass 0 if no movement is required. vy:Lateral velocity (required). Pass 0 if no movement is required. yaw_rate:Angular velocity around the Z-axis (required). Pass 0 if no movement is required.
Return Value	Type：uint32_t 0:normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	This state can only be entered when the robot is in a standing position, allowing control of the dog to move at specified speeds. vx range（-3m/s ~ -0.05m/s; 0.05m/s ~ 3m/s） vy range（-1.0m/s ~ -0.1m/s;0.1m/s ~ 1.0m/s） yaw_rate range（-3.0rad/s ~ -0.02rad/s;0.02rad/s ~ 3.0rad/s）

1.7. Jump

Function Name	jump()
Function Overview	Control the robot to jump vertical
parameter	None
Return Value	Type：uint32_t 0:normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	The robot cannot directly switch to this state while moving.

1.8. Forward Jump

Function Name	frontJump()
Function Overview	Control the robot to jump forward
parameter	None
Return Value	Type：uint32_t 0:normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	The robot cannot directly switch to this state while moving.

1.9. Back Flip

Function Name	backflip()
Function Overview	Control the robot to back flip
parameter	None
Return Value	Type：uint32_t 0:normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	The robot cannot directly switch to this state while moving.

1.10. Shake Hand

Function Name	**shakeHand()**
Function Overview	Control the robot to shake hand
parameter	None
Return Value	Type：uint32_t 0:normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	The robot cannot directly switch to this state while moving.

1.11. Two Leg Stand

Function Name	**twoLegStand(float vx, float yaw_rate)**
Function Overview	Control the robot to stand
parameter	vx:Forward velocity (required). Pass 0 if no movement is required. yaw_rate:Angular velocity around the Z-axis (required). Pass 0 if no movement is required.
Return Value	Type：uint32_t 0:normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	The robot cannot directly switch to this state while moving. To exit the two-leg standing state, you must call cancelTwoLegStand(). No other interfaces will be responded to during this transition. You can control the robot to move at specified speeds through parameters. vx range（-0.5~-0.2 / 0.2~0.5m / s） yaw_raterange（-1~-0.2 / 0.2~1.0 rad / s）

1.12. Exit Two Leg Stand

Function Name	**cancelTwoLegStand()**
Function Overview	Control the robot to exit standing
parameter	None
Return Value	None
Note	The robot exits the two-leg standing state. The robot can transition into this state only from the two-leg standing position.

1.13. Attitude Control

Function Name	**attitudeControl(float roll_vel, float pitch_vel,float yaw_vel, float height_vel)**
Function Overview	Control the robot to twist in place and change the height of the body
parameter	roll_vel:Angular velocity around the X-axis (required). Pass 0 if no movement is required. pitch_vel: Angular velocity around the Y-axis (required). Pass 0 if no movement is required. yaw_vel: Angular velocity around the Z-axis (required). Pass 0 if no movement is required. height_vel: Vertical velocity (required). Pass 0 if no movement is required.
Return Value	Type：uint32_t 0:normal state 0x3012:Motor data loss 0x3010:Motor disabled 0x3011:Motor fault 0x3009:Motor angle limit exceeded 0x3007:State machine switching failed 0x3013:Speed command too large
Note	The robot can transition into this state from a standing position, allowing control to twist in place at a specified speed while changing body height. yaw_vel, pitch_vel, roll_velrange（-0.5rad/s~0.5rad/s）height_velrange（-0.5m/s-0.5m/s）

1.14.Get Quaternion of Posture

Function Name	**getQuaternion()**
Function Overview	Obtain the current quaternion of the robot's posture
parameter	None
Return Value	A vector consisting of 4 elements [w, x, y, z] corresponds to the four components of a quaternion.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

1.15. Get RPY

Function Name	**getRPY()**
Function Overview	Obtain the Euler angles (roll angle, pitch angle, yaw angle) of the current posture of the robot.
parameter	None
Return Value	A vector consisting of 3 elements [roll, pitch, yaw], with units in radians (rad), respectively representing the rotation angles around the X, Y, and Z axes.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

1.16. Get Acceleration

Function Name	**getBodyAcc()**
Function Overview	Obtain the acceleration data
parameter	None
Return Value	A vector consisting of 3 elements [ax, ay, az], with units of m/s^2, respectively correspond to the acceleration in the X, Y, and Z axis directions.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

1.17. Get Angular Velocity

Function Name	**getBodyGyro()**
Function Overview	Obtain the angular velocity data of the robot's body (in the body coordinate system).
parameter	None
Return Value	It is a vector with 3 elements [gx, gy, gz], with units in rad/s, corresponding to the angular velocities along the X, Y, and Z axes, respectively.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

1.18. Get Position

Function Name	**getPosition()**
Function Overview	Obtain the position coordinates in the origin coordinate system.
parameter	None
Return Value	A vector with 3 elements [x, y, z], with units of meters (m), represents the current position.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

1.19. Get World Velocity

Function Name	**getWorldVelocity()**
Function Overview	Obtain the moving speed in the coordinate system centered at the origin
parameter	None
Return Value	A vector with 3 elements [vx, vy, vz], with units of meters per second (m/s), respectively representing the velocities in the X, Y, and Z directions of the upper power origin coordinate system.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

1.20.Get Body Velocity

Function Name	**getBodyVelocity()**
Function Overview	Obtain the moving speed of the robot in the body coordinate system.
parameter	None
Return Value	A vector with 3 elements [vx, vy, vz], with units of m/s, respectively represents the velocities along the X, Y, and Z axes of the vehicle's coordinate system.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

1.21. Get Battery Power

Function Name	**getBatteryPower()**
Function Overview	Get current battery power
parameter	None
Return Value	Type: uint32_t range: 0-100
Note	Return Valuerange 0-100, 0 lack of battery, 100 full battery

1.22. Get Control Mode

Function Name	**getCurrentCtrlmode()**
Function Overview	Get current control mode of robot
parameter	None
Return Value	Type： uint32_t 0: The device is not enabled. 1:Standing State / Greeting State 10:Free Motor State triggered when the device is lying down 18:Moving State 21:Action State (Posture Mode, Jump Mode, Two-Legged Standing, etc.) 51:Lying Down State.
Note	None

1.23. Get Joint Angle of Abad

Function Name	**getLegAbadJoint()**
Function Overview	Get Joint Angle of Abad
parameter	None
Return Value	A vector consisting of 4 elements, corresponding to the Abad joint angles of the right anterior, left anterior, right posterior, and left posterior respectively (unit: rad).
Note	unit： rad. The robot's forward direction is X, left is Y, and vertically upward is Z

1.24. Get Joint Angle of Hip

Function Name	**getLegHipJoint()**
Function Overview	Get Joint Angle of Hip
parameter	None
Return Value	A vector containing 4 elements, corresponding to the Hip joint angles (unit: rad) of the Right Front , Left Front , Right Rear , and Left Rear legs respectively.
Note	Unit: rad/s. For the robot, the direction forward is X, the left side is Y, and the vertical upward direction is Z.

1.25. Get Joint Angle of Knee

Function Name	getLegKneeJoint()
Function Overview	Get Joint Angle of Knee
parameter	None
Return Value	A vector containing 4 elements, corresponding to the Knee joint angles of the Right Front , Left Front , Right Rear , and Left Rear legs respectively.(unit: rad)
Note	Unit: rad/s. For the robot, the direction forward is X, the left side is Y, and the vertical upward direction is Z.

1.26. Get Angular Velocity of Abad Joint

Function Name	getLegAbadJointVel()
Function Overview	Get Angular Velocity of Abad Joint
parameter	None
Return Value	A vector containing 4 elements, with units in rad/s, corresponding to the joint angles of the Right Front , Left Front , Right Rear , and Left Rear legs respectively.
Note	Unit: rad/s. For the robot, the direction forward is X, the left side is Y, and the vertical upward direction is Z.

1.27. Get Angular Velocity of Hip Joint

Function Name	getLegHipJointVel()
Function Overview	Get Angular Velocity of Hip Joint
parameter	None
Return Value	A vector containing 4 elements, with units in rad/s, corresponding to the hip joint angles of the Right Front , Left Front , Right Rear , and Left Rear legs respectively.
Note	Unit: rad/s. For the robot, the direction forward is X, the left side is Y, and the vertical upward direction is Z.

1.28. Get Angular Velocity of Knee Joint

Function Name	getLegKneeJointVel()
Function Overview	Get Angular Velocity of Knee Joint
parameter	None
Return Value	A vector containing 4 elements, with units in rad/s, corresponding to the joint angles of the Right Front , Left Front , Right Rear , and Left Rear legs respectively.
Note	Unit: rad/s. For the robot, the direction forward is X, the left side is Y, and the vertical upward direction is Z.

1.29. Get Joint Torque of Abad

Function Name	getLegAbadJointTorque()
Function Overview	Get Joint Torque of Abad
parameter	None
Return Value	A vector containing 4 elements, typically with units in N・m, corresponding in order to the joint torques.
Note	Typically measured in N·m, corresponding to the knee joint torques of the Right Front , Left Front , Right Rear , and Left Rear legs respectively.

1.30. Get Joint Torque of Hip

Function Name	getLegHipJointTorque()
Function Overview	Get Joint Torque of Hip
parameter	None
Return Value	A vector containing 4 elements, typically with units in N・m, corresponding in order to the joint torques.
Note	Typically measured in N·m, corresponding to the knee joint torques of the Right Front , Left Front , Right Rear , and Left Rear legs respectively.

1.31. Get Joint Torque of Knee

Function Name	getLegKneeJointTorque()
Function Overview	Get Joint Torque of Knee
parameter	None
Return Value	A vector containing 4 elements, typically with units in N・m, corresponding in order to the joint torques.
Note	Typically measured in N·m, corresponding to the knee joint torques of the Right Front , Left Front , Right Rear , and Left Rear legs respectively.

2.LowLevel Function Introduction

class lowlevel

- This class is used for interaction with the motor, including sending motor commands and obtaining motor status.

> 🎨 The supported frequency for the device status acquisition interface in motion control is: LowLevel supports 500 Hz.
>
> The supported frequency for the request interface in motion control is: data requests must be sent at 500 Hz to ensure stable operation of the device.

2.1 Establish Connection

Function Name	initRobot(std::string & local_ip, int local_port, string& dog_ip = "192.168.234.1")
Function Overview	Establish communication with the robot dog. The default IP on the robot dog side is “192.168.234.1”. If the robot's IP has been changed, the new IP must be passed through this interface.
parameter	local_ip:User host IP address (required) local_port:User host port (required) dog_ip:Robot IP address (optional)
Return Value	None
Note	Failed to establish connection. Terminal output failure logs

2.2. Check Connection

Function Name	checkConnection()
Function Overview	Check the connection status of robot
parameter	None
Return Value	Type：bool true:Connection successful false:Connection fail
Note	Can perform heart rate detection

2.3 Send Motor Command

Function Name	sendMotorCmd(motorCmd& cmd)
Function Overview	Send Motor Command
parameter	motorCmd
Return Value	return0 when sucess，return-1 when failure
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

- Cmd Data Protocol

```
struct alignas(4) motorCmd  {

  float q_des_abad[4] = {0.0, 0.0, 0.0, 0.0}; // Angle of joint A
  float q_des_hip[4] = {0.0, 0.0, 0.0, 0.0};  // Angle of joint H
  float q_des_knee[4] = {0.0, 0.0, 0.0, 0.0}; // Angle of joint K

  float qd_des_abad[4] = {0.0, 0.0, 0.0, 0.0}; // A-joint angular velocity command
  float qd_des_hip[4] = {0.0, 0.0, 0.0, 0.0};  // H-joint angular velocity command
  float qd_des_knee[4] = {0.0, 0.0, 0.0, 0.0}; // K-joint angular velocity command

```

```
    float kp_abad[4] = {0.0, 0.0, 0.0, 0.0}; // Kp value of joint A
    float kp_hip[4] = {0.0, 0.0, 0.0, 0.0};  // Kp value of joint H
    float kp_knee[4] = {0.0, 0.0, 0.0, 0.0}; // Kp value of joint K

    float kd_abad[4] = {0.0, 0.0, 0.0, 0.0}; // Kd value of joint A
    float kd_hip[4] = {0.0, 0.0, 0.0, 0.0};  // Kd value of joint H
    float kd_knee[4] = {0.0, 0.0, 0.0, 0.0}; // Kd value of joint K

    float tau_abad_ff[4] = {0.0, 0.0, 0.0, 0.0}; // A-joint torque command
    float tau_hip_ff[4] = {0.0, 0.0, 0.0, 0.0};  // H-joint torque command
    float tau_knee_ff[4] = {0.0, 0.0, 0.0, 0.0}; // K-joint torque command

};
```

2.4. Get Quaternion of Posture

Function Name	**getQuaternion()**
Function Overview	Obtain the current quaternion of the robot's posture
parameter	None
Return Value	A vector consisting of 4 elements [w, x, y, z] corresponds to the four components of a quaternion.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

2.5. Get RPY

Function Name	**getRPY()**
Function Overview	Obtain the Euler angles (roll angle, pitch angle, yaw angle) of the current posture of the robot.
parameter	None
Return Value	A vector containing 3 elements [roll, pitch, yaw] , with units in radians (rad), corresponding to the rotation angles around the X, Y, ar axes respectively.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

2.6. Get Acceleration

Function Name	**getBodyAcc()**
Function Overview	Obtain the acceleration data of the robot's body (in the body coordinate system).
parameter	None
Return Value	A vector containing 3 elements [ax, ay, az] , with units in m/s^2, corresponding to the accelerations along the X, Y, and Z axes respectively.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

2.7. Get Angular Velocity

Function Name	getBodyGyro()
Function Overview	Obtain the angular velocity data of the robot's body (in the body coordinate system).
parameter	None
Return Value	It is a vector with 3 elements [gx, gy, gz], with units in rad/s, corresponding to the angular velocities along the X, Y, and Z axes, respectively.
Note	The robot's forward direction is X, left is Y, and vertically upward is Z.

2.8. Get Motor State

Function Name	getMotorState()
Function Overview	Obtain the motor data of the robot
parameter	None
Return Value	return the motor data
Note	unit rad

- MotorState

```
struct alignas(4) motorState {
  // 关节顺序 FL,FR,RL,RR
  float q_abad[4] = {0.0, 0.0, 0.0, 0.0}; // Angle of joint A
  float q_hip[4] = {0.0, 0.0, 0.0, 0.0};  // Angle of joint H
  float q_knee[4] = {0.0, 0.0, 0.0, 0.0}; // Angle of joint K


  float qd_abad[4] = {0.0, 0.0, 0.0, 0.0}; // A-joint angular velocity
  float qd_hip[4] = {0.0, 0.0, 0.0, 0.0};  // H-joint angular velocity
  float qd_knee[4] = {0.0, 0.0, 0.0, 0.0}; // K-joint angular velocity


  float tau_abad_fb[4] = {0.0, 0.0, 0.0, 0.0};// A-joint torque feedback
  float tau_hip_fb[4] = {0.0, 0.0, 0.0, 0.0};  // H-joint torque feedback
  float tau_knee_fb[4] = {0.0, 0.0, 0.0, 0.0}; // K-joint torque feedback
};
```

2.9 Existence of Motor Data

Function Name	haveMotorData()
Function Overview	Check whether motor data has been acquired.
parameter	None
Return Value	returntrue or false
Note	Check whether motor data has been received before sending motor commands.

Chapter V: FAQ

Q1：connect time out

After running the SDK program, there was a connection timeout. The prompt message was：

Code block

```
client bind success
connect time out
error async receive from
```

1. Please modify the target_ipparameter according to SDK
2. If communication with the device does not use Wi-Fi direct connection, also modify the SDK_CLIENT_IP parameter . In non-Wi-Fi direct connection mode, there is a probability that the device may fail to bind the SDK for motion control. In such cases, please modify the parameter by referring to Wired Connection Binding IP Failure

Q2：Wired Connection Binding IP Failure

After configuring `SDK_CLIENT_IP` , if SDK control is performed via the `wlan0` or `eth0` network, the device’s network may not have fully initialized upon startup, and the corresponding network interface may not yet have an assigned IP address, resulting in a failure to bind the IP address.

The specific symptom is that after running the SDK, a `connect time out` message appears. After logging into the device’s main controller, executing the command

`sudo lsof -i -P -n -c mc_ctrl | grep 43997`

reveals that the motion control port is not bound.

This issue can be resolved by modifying the configuration file `/etc/systemd/system/robot-launch.service` on the main controller. Add the following line in the `Service` section:

`Environment="ROBOT_NET_INTERFACES=wlan0"`

Replace `wlan0` with the corresponding network interface name. If using a wired network, change it to `eth0`.

The complete configuration file is as follows:

Code block

```
[Unit]
Description=Run robot-launch at startup
After=network.target

[Service]
ExecStart=robot-launch server
ExecStop=/usr/local/bin/robot_launch_stop.sh
TimeoutStopSec=5s
Restart=always
User=root
Environment="ROBOT_HOME=/home/firefly"
Environment="ROBOT_LOG_DIR=/userdata/log"
Environment="ROBOT_NET_INTERFACES=wlan0"

[Install]
WantedBy=multi-user.target
```

⚽ The parameter will take effect after the device is restarted.

👍 Configuring this parameter may cause a chance that the robot program fails to start!

After `ROBOT_NET_INTERFACES` is configured, the program will check whether the configured network interface is assigned an IP address. If the network interface never obtains an IP, it will result in the robot program being unable to start.

Q3：When using wlan0 to connect to an environment Wi-Fi, the device does not automatically reconnect after each restart.

There is a conflict between the onboard AP and the wlan network card on the device. Some versions include a feature that clears automatic connections. To disable this feature and allow automatic reconnection, execute the following command:

```
sudo systemctl stop networkmanager-cleanup.service      //Turn off the network clearing service
sudo systemctl disable networkmanager-cleanup.service   //Disable Network Cleanup Service
```

Q4: Python Import Failed

Check if the script has added `lib/<model>/<arch>` to `sys.path`; Python version must match the extension modules.

Q5: C++ Connect Failed?

Checkif `.so` consists in `lib/<model>/<arch>` ; make sure CMake could dectets (`x86_64` / `aarch64`)

Q6: What is the supported frequency for obtaining device status ?

Lowlevel supports 500 Hz, while Highlevel supports 50 Hz.

Q7: What frequency is the control interface in the SDK ?

The control data can be transmitted at up to 500 Hz, but generally, 20–50 Hz is sufficient to ensure stable operation of the device.

Q8: Is the SDK program thread-safe?

Yes, the SDK program is thread-safe.

Q9:Does the SDK have communication detection?

The motion control program monitors whether SDK data is continuously received. If no data is received for 3 seconds, the SDK communication is considered interrupted, and the device will automatically switch to damping mode and lower itself.

Q10: When starting the motion control program via a service inside the device, how to prevent the device's own program from not starting?

In the service configuration, the `After` parameter can be set to `robot-launch`, as the `robot-launch` service will start the device's own program.

Q11: Can joystick remote control and SDK remote control be switched simultaneously?

In the current version, the SDK has higher priority than joystick remote control, so simultaneous switching is not possible.

Q12: Is the acquired IMU data raw data?

Yes, the acquired IMU data is raw data.

Q13: Why does the acquired IMU data seem to have errors?

The IMU data inside the device is influenced by the product model, and its accuracy may have inherent errors. If higher IMU precision is required, a separate high-precision IMU must be purchased.

Q14: After using SDK control, why does the device perform 'push-ups' on the spot when sending a stand command via joystick remote control?

If the SDK_CLIENT_IP paramete has been configured, the motion control program will bind to the corresponding IP upon startup. If the IP does not exist, this phenomenon may occur.

Q15:What is the rotation order when converting quaternions to Euler angles?

The Euler angle order is ZYX, meaning rotation around the Z-axis first, then the Y-axis, and finally the X-axis.

Q16: Can Highlevel and Lowlevel be used simultaneously?

No, Highlevel and Lowlevel cannot be used simultaneously. When switching between the two interfaces, sufficient time must be allowed for the device ports to be released.

Q17: Why can the SDK no longer control the device after a motion control program upgrade?

After upgrading the motion control program, related configurations may be overwritten. You need to reconfigure theSDK_CONFIG and SDK_CLIENT_IPparameter。

Q18: Is there a ROS interface available?

Currently, there is no ROS interface.

Q19: When using the SDK, the program gets stuck at the `initRobot` function and does not proceed. Why?

The SDK depends on Boost version 1.74.0. If an older version is used, there may be compatibility issues due to changes in data structures.

Q20:Is it supported to deploy control logic on the robot itself?

Deploying control logic directly on the robot is not recommended. The total internal computing capacity of the robot is 6 TOPS, and there is no surplus computing power available.